

BCIS technical documentation

System overview

The three office PCs run Electron clients. A central Fastify API is the only application component allowed to use PostgreSQL. Attachments and backups are server-owned. React communicates through a narrow typed preload request bridge; a browser development mode uses the same HTTP API.

Source map

source/server/app.ts: HTTP routes, Zod validation, authorization and workflow orchestration.

source/server/finance.ts: row locks, billing, payment allocation, reversal, ledger reconstruction.

source/server/db.ts: PostgreSQL pool, transaction wrapper, Drizzle connection, audit insert helper.

source/server/auth.ts: salted script password hashing and hashed session tokens.

source/server/reports.ts: report queries and PDF/XLSX rendering.

source/server/backup.ts: archive creation, attachment snapshot and manifest verification.

source/shared/domain.ts: exact decimal parsing, allocation, aging and role permission seed.

source/shared/bridge.ts: renderer-to-main request/response contract.

source/electron/: isolated Electron main and preload.

source/ui/: connected React screens and visual design tokens.

database/migrations/: versioned schema, indexes and immutable-history triggers.

scripts/: local database, restore drill, UI/workflow checks, deliverable generation.

Data dictionary

users: unique username, display name, salted password hash, active state.

roles / permissions / role_permissions / user_roles: normalized access control.

sessions: token hash, user and expiry; raw bearer tokens are never persisted.

subscribers: unique account, synthetic identity/contact/address, area/collector, billing/due days, status and notes.

service_plans: unique code, Internet/Cable/Combo, price in centavos, optional speed/channel attributes.

service_accounts: unique service number, subscriber, plan, installation address, agreed rate, activation/billing-start dates and operational state.

billing_cycles: calendar billing period and generating actor.

invoices: unique number, service/period uniqueness, immutable amount and due date, lifecycle status.

invoice_items: item description and centavo amount.

adjustments: append-only invoice debit/credit, reason, actor and timestamp.

payments: unique receipt, subscriber, integer amount, method/reference, actor, optional batch/proof, idempotency key and reversal flag.

payment_allocations: payment-invoice many-to-many links and centavo amounts; reversed-payment allocations are excluded from balances.

payment_proofs: normalized unique reference, sender, amount, generated attachment filename, review status/actor/time/reason.

payment_reversals: original payment, actor, reason and time; one reversal per payment.

collection_areas / collectors: route and collector directory.

collection_batches / batch_accounts: collector, area, lifecycle, expected balances and route membership.

collector_remittances: immutable expected cash, remitted cash, difference, actor and reason.

service_events / suspension_records / reconnection_records: operational history, approvals and technician completion.

audit_logs: append-only actor/action/entity/reason/new-value records.

application_settings: JSON service policy.

backup_history: archive filename, SHA-256 checksum and verification status.

Relationships

Subscriber -> many service accounts -> many invoices -> many invoice items.

Invoice <-> payment through payment_allocations.

Payment -> optional proof and at most one reversal.

Collector -> many batches -> many batch accounts.

Batch -> at most one immutable remittance record.

Service -> many operational events, suspensions and reconnections.

User <-> role <-> permission through join tables.

Transaction boundaries

Financial posting locks the subscriber row first, then performs all mutations in one PostgreSQL transaction.

Payment and billing allocation both honor this lock. PostgreSQL sequences allocate unique numbers under concurrency. Constraint violations roll back the transaction. On network retry, payment idempotency returns the same receipt instead of creating a duplicate.

Collection posting also locks the batch, preventing a payment racing with submission/remittance. Reversals cannot silently change submitted or reconciled batches. History triggers disallow destructive financial updates.

API and IPC

All operational paths start /api/. Login creates an eight-hour bearer session; logout deletes it. Protected routes validate active users and permission rows server-side. GET retrieves records and POST performs explicit actions. Zod validates mutation payloads, date formats, IDs, amounts and reasons. Queries use parameter placeholders.

Primary routes: /subscribers, /subscribers/:id, /services, /billing/generate, /invoices, /payments, /payments/:id/reverse, /proofs, /proofs/:id/upload, /proofs/:id/review, /batches, /batches/:id/remit, /batches/:id/transition, /

receivables, /dashboard, /reports/:type, /users, /audit and /backups.

Electron exposes request({path, method, token, body, binary, upload}) and a report-only Save dialog action. Main validates the trusted renderer and API path, bounds uploads and performs HTTP requests. No direct SQL, arbitrary filesystem, command or IPC channel is exposed. Renderer Node integration is disabled, context isolation and sandboxing are enabled.

Deployment and limitations

See deployment-guide.md for LAN setup, Windows build and offline safe restore. See requirements-and-decisions.md for financial policy and recommended-stack deviations. See acceptance-status.md for actual evidence and unverified production-scale/Windows checks. Reports are operational management reports. GCash evidence is manually checked; no payment gateway or social-platform access is assumed.